

Chaztla

Documentación Técnica del Proyecto

Aplicación web de chat con inteligencia artificial

rct-chaztia-fe · versión 0.0.0

Índice

Índice.....	2
1. Descripción general.....	3
1.1 Objetivos del proyecto.....	3
2. Arquitectura y stack tecnológico	3
2.1 Stack tecnológico	3
2.2 Diagrama de flujo (alto nivel)	3
3. Estructura de carpetas.....	5
4. Frontend (React).....	6
4.1 App.tsx — componente principal.....	6
Estado principal	6
Tipos principales	6
4.2 Flujo de envío de un mensaje	6
4.3 Adjuntar archivos	6
4.4 Renderizado de Markdown.....	7
4.5 Persistencia.....	7
5. Interfaz visual y estilos.....	8
5.1 Sistema de theming.....	8
5.2 Scrollbar personalizado.....	8
5.3 Efecto de brillo ("glow") en el input inicial	8
5.4 Indicador de "escribiendo"	8
5.5 Layout responsivo	8
6. Backend: función serverless (api/chat.ts).....	9
6.1 Contrato de la API	9
6.2 Modelo utilizado	9
6.3 Variables de entorno requeridas.....	9
7. Despliegue en Vercel	10
7.1 Configuración (vercel.json)	10
7.2 Pasos de despliegue.....	10
7.3 Desarrollo local.....	10
8. Consideraciones y limitaciones conocidas.....	11
9. Glosario	11

1. Descripción general

ChaztIa es una aplicación web de chat conversacional que permite al usuario interactuar con un modelo de lenguaje (IA) a través de una interfaz simple, moderna y responsiva. La aplicación corre completamente en el navegador (frontend en React) y delega el procesamiento de las conversaciones a un modelo de lenguaje alojado en Groq mediante una función serverless.

El proyecto está pensado como una single-page application (SPA) sin backend propio ni base de datos: toda la conversación, las preferencias visuales y los archivos adjuntos existen únicamente durante la sesión del navegador.

1.1 Objetivos del proyecto

- Ofrecer una interfaz de chat conversacional clara y agradable, en español.
- Permitir personalización básica (nombre de usuario, foto de perfil, tema claro/oscuro).
- Soportar el envío de archivos e imágenes como contexto adicional para la IA.
- Renderizar las respuestas del modelo con formato enriquecido (Markdown, bloques de código).
- Ser desplegable de forma sencilla en Vercel, sin infraestructura adicional.

2. Arquitectura y stack tecnológico

La aplicación sigue una arquitectura cliente-servidor mínima: un cliente SPA en React que se comunica con una única función serverless (api/chat.ts) que a su vez reenvía la conversación a la API de Groq (compatible con el formato de OpenAI Chat Completions).

2.1 Stack tecnológico

Capa	Tecnología	Rol
Framework UI	React 19	Renderizado de la interfaz y manejo de estado
Lenguaje	TypeScript	Tipado estático en frontend y función serverless
Build tool	Vite 8	Servidor de desarrollo y bundling de producción
Estilos	Tailwind CSS v4	Utilidades CSS, theming claro/oscuro
Componentes UI	shadcn / Base UI	Primitivas accesibles (Dialog, Input, Avatar, etc.)
Markdown	react-markdown + remark-gfm	Renderizado de texto enriquecido y bloques de código
Iconos	lucide-react	Set de iconos SVG
Backend	Vercel Serverless Functions (Node.js)	Proxy hacia la API de Groq
Proveedor de IA	Groq API (modelo openai/gpt-oss-120b)	Generación de respuestas del asistente
Hosting	Vercel	Despliegue de frontend y funciones

2.2 Diagrama de flujo (alto nivel)



React SPA (src/App.tsx)

```
| fetch POST /api/chat { messages: [...] }
```

▼

Función serverless (api/chat.ts, Vercel)

```
| fetch POST https://api.groq.com/openai/v1/chat/completions
```

▼

Groq API (modelo openai/gpt-oss-120b)

```
| respuesta { choices[0].message.content }
```

▼

React SPA → se agrega el mensaje "assistant" y se renderiza con Markdown

3. Estructura de carpetas

```
rct-chaztla-fe/
├─ api/
│   └─ chat.ts          Función serverless (proxy a Groq)
├─ public/
│   ├── favicon.png     Ícono de la aplicación
│   └─ favicon.svg
├─ src/
│   ├── main.tsx        Punto de entrada de React
│   ├── App.tsx         Componente raíz: toda la lógica del chat
│   ├── index.css       Tema, variables de color, animaciones
│   ├── components/
│   │   ├── chat-markdown.tsx  Render de Markdown en los mensajes
│   │   └─ ui/                 Componentes shadcn (Button, Input, Card, Dialog...)
│   └─ lib/
│       └─ utils.ts          Helper cn() (clsx + tailwind-merge)
├─ index.html            HTML raíz, título y favicon
├─ vite.config.ts        Configuración de Vite (alias @, plugins)
├─ vercel.json           Configuración de despliegue en Vercel
├─ package.json
└─ .env.example          Variables de entorno requeridas
```

4. Frontend (React)

4.1 App.tsx — componente principal

Todo el estado y la lógica de la aplicación vive en el componente App (src/App.tsx). No se utiliza una librería externa de manejo de estado (Redux, Zustand, etc.): se gestiona con useState y useRef de React.

Estado principal

Estado	Tipo	Descripción
messages	Message[]	Historial de la conversación actual (usuario/asistente)
input	string	Texto actualmente escrito en el campo de mensaje
loading	boolean	true mientras se espera la respuesta de la IA
started	boolean	true desde que se envía el primer mensaje
pendingAttachments	Attachment[]	Archivos adjuntados pero aún no enviados
theme	light dark	Tema visual activo (persistido en localStorage)
userName	string	Nombre que la IA usa para dirigirse al usuario (persistido)
avatar	string null	Foto de perfil en base64 (persistida)

Tipos principales

```
type Attachment = {
  id: string; name: string; type: string; size: number;
  dataUrl: string; textContent?: string;
}

type Message = {
  role: "user" | "assistant" | "system";
  content: string;
  attachments?: Attachment[];
}
```

4.2 Flujo de envío de un mensaje

- 1. **Captura:** el usuario escribe en el <Input> y/o adjunta archivos con el botón de clip.
- 2. **sendMessage():** construye un Message con el texto y los adjuntos pendientes, lo agrega a messages y limpia el input.
- 3. **buildContentForApi():** para cada adjunto de texto (código, .txt, .md, .json, etc.) inserta su contenido como bloque de código en el mensaje enviado a la API; para binarios (imágenes, PDF) agrega una nota indicando que se adjuntó un archivo, ya que el modelo actual no procesa imágenes.
- 4. **fetch("/api/chat"):** se envía la lista completa de mensajes (incluyendo un mensaje "system" con el nombre del usuario, si fue configurado).
- 5. **Respuesta:** se agrega un nuevo Message con role "assistant" al historial y se renderiza con ChatMarkdown.

4.3 Adjuntar archivos

El botón de clip abre un selector de archivos (múltiple). Por cada archivo se genera un Attachment mediante `fileToAttachment()`: se lee como Data URL (para previsualización/descarga) y, si el archivo es de tipo texto/código (detectado por MIME type o extensión), también se lee su contenido como texto plano.

Límite	Valor
Archivos por mensaje	Máximo 5 (MAX_ATTACHMENTS)
Tamaño por archivo	Máximo 8 MB (MAX_ATTACHMENT_SIZE)
Contexto de texto enviado a la IA	Máximo ~12.000 caracteres por archivo (truncado)

Los adjuntos e imágenes se muestran dentro de la burbuja del mensaje: las imágenes como miniatura, y otros archivos como un enlace descargable con nombre y tamaño.

4.4 Renderizado de Markdown

El componente ChatMarkdown (`src/components/chat-markdown.tsx`) usa react-markdown con el plugin remark-gfm (tablas, listas de tareas, etc.). Personaliza el renderizado de: párrafos, listas, enlaces (se abren en una nueva pestaña) y código.

El código inline se muestra con fondo diferenciado; los bloques de código (```lenguaje`) se muestran dentro de una caja con encabezado indicando el lenguaje y scroll horizontal si el contenido es muy ancho.

4.5 Persistencia

La aplicación distingue explícitamente entre datos persistentes y datos de sesión:

- **Persistente (localStorage):** nombre de usuario, tema (claro/oscuro) y foto de perfil. Sobreviven a recargar la página.
- **Solo en memoria (useState):** los mensajes de la conversación y los archivos adjuntos. Se pierden al recargar la página o cerrar la pestaña, por diseño.

5. Interfaz visual y estilos

5.1 Sistema de theming

Los colores se definen como variables CSS (custom properties) en `src/index.css` usando el espacio de color OKLCH, con un bloque `:root` para el tema claro y un bloque `.dark` para el tema oscuro. Tailwind v4 mapea estas variables a utilidades (`bg-background`, `text-foreground`, etc.) mediante `@theme inline`.

5.2 Scrollbar personalizado

Se estiliza la barra de scroll (`scrollbar-width/scrollbar-color` para Firefox y `::-webkit-scrollbar-*` para navegadores basados en Chromium) usando las mismas variables de color del tema, por lo que se adapta automáticamente entre modo claro y oscuro.

5.3 Efecto de brillo ("glow") en el input inicial

Antes de enviar el primer mensaje, el campo de texto muestra un brillo animado alrededor (gradiente cónico multicolor que rota, similar al de Gemini), implementado con `@property --gemini-angle` y una animación CSS. El brillo se intensifica al enfocar el input y desaparece por completo una vez que existe al menos un mensaje en la conversación, para no distraer durante la conversación activa.

5.4 Indicador de "escribiendo"

Mientras se espera la respuesta de la IA se muestra una burbuja con el texto "Escribiendo" seguida de tres puntos animados (thinking-dot) que laten con una animación de escala y opacidad, con retrasos escalonados entre cada punto para simular una animación de "pensando".

5.5 Layout responsivo

El contenedor principal usa un ancho máximo (`max-w-4xl`) centrado. El área de mensajes usa flexbox con `overflow-y-auto` para el scroll, y el campo de entrada se ubica en flujo normal debajo (no absoluto), evitando que se superponga con el final del scroll de mensajes.

6. Backend: función serverless (api/chat.ts)

La única pieza de backend del proyecto es una función serverless de Vercel (Node.js) ubicada en api/chat.ts. Su única responsabilidad es actuar como proxy seguro hacia la API de Groq, evitando exponer la API key en el navegador.

6.1 Contrato de la API

Campo	Detalle
Método	POST
Ruta	/api/chat
Body	{ "messages": [{ "role": "system" "user" "assistant", "content": string }] }
Respuesta OK (200)	{ "reply": string }
Respuesta error	{ "error": string } (400, 405 o 500)

6.2 Modelo utilizado

Se utiliza el modelo openai/gpt-oss-120b servido por Groq, a través del endpoint compatible con OpenAI Chat Completions (<https://api.groq.com/openai/v1/chat/completions>). Este modelo es de solo texto: no procesa contenido de imágenes.

6.3 Variables de entorno requeridas

Variable	Descripción
GROQ_API_KEY	Clave de API de Groq, requerida por la función serverless. Nunca debe exponerse en el frontend.

7. Despliegue en Vercel

7.1 Configuración (vercel.json)

```
{
  "framework": "vite",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "installCommand": "npm install"
}
```

No se define ninguna regla de rewrite/SPA fallback: la aplicación no usa enrutamiento del lado del cliente (no hay React Router), por lo que no es necesaria. Se optó deliberadamente por no incluirla, ya que interfiere con vercel dev al proxyear todas las rutas a través del servidor de desarrollo de Vite.

7.2 Pasos de despliegue

- **1.:** Configurar la variable de entorno GROQ_API_KEY en el proyecto de Vercel (Dashboard → Settings → Environment Variables, o `npx vercel env add GROQ_API_KEY`).
- **2.:** Ejecutar `npx vercel --prod` desde la raíz del proyecto (o conectar el repositorio Git para despliegues automáticos).
- **3.:** Vercel detecta automáticamente el framework Vite y despliega la función `api/chat.ts` como Serverless Function.

7.3 Desarrollo local

Comando	Descripción
<code>npm run dev</code>	Levanta el servidor de desarrollo de Vite (solo frontend, sin funciones serverless).
<code>npx vercel dev</code>	Levanta un entorno local que simula Vercel, incluyendo la función <code>api/chat.ts</code> .
<code>npm run build</code>	Genera el build de producción en <code>dist/</code> .
<code>npm run preview</code>	Sirve el build de producción localmente.
<code>npm run lint</code>	Ejecuta ESLint sobre el proyecto.

8. Consideraciones y limitaciones conocidas

- El modelo de IA utilizado (gpt-oss-120b vía Groq) no tiene capacidad de visión: las imágenes adjuntas se muestran en la interfaz y quedan disponibles para el usuario, pero la IA solo recibe una nota indicando que se adjuntó una imagen, no su contenido visual.
- No existe persistencia de conversaciones: al recargar la página o cerrar la pestaña se pierde el historial completo y los archivos adjuntos, por diseño (no hay backend con base de datos).
- No hay autenticación de usuarios: la app es de uso libre, sin cuentas ni login.
- La clave de API de Groq se mantiene únicamente en el servidor (variable de entorno de Vercel) y nunca se expone al cliente.
- El proyecto no usa React Router; toda la aplicación es una única vista.

9. Glosario

- **SPA:** Single Page Application: aplicación que se ejecuta en una sola página HTML, sin recargas completas.
- **Serverless Function:** Función que se ejecuta bajo demanda en la infraestructura de Vercel, sin servidor propio persistente.
- **Data URL:** Representación de un archivo binario codificado en base64 dentro de una cadena de texto (data:tipo;base64,...).
- **Markdown:** Lenguaje de marcado ligero usado para dar formato a texto (negritas, listas, código, etc.).